MLP Viva: Extra Unique Questions & Answers — Part 3

Remaining questions extracted from full page-by-page scan of all student viva submissions

CONCEPTUAL CLARITY QUESTIONS

What is the difference between a metric and a loss function?

A loss function is used DURING training to optimise the model — it must be differentiable so gradients can be computed (e.g., cross-entropy, MSE, hinge loss). A metric is used AFTER training to evaluate model performance for humans — it does not need to be differentiable (e.g., accuracy, F1-score, AUC-ROC). Example: you train a classifier with log-loss but evaluate it using F1-score. The same function can be both (e.g., MSE is both a loss and a regression metric), but they serve different roles.

What is 0-centring (zero-centring) and why does it matter?

Zero-centring (mean subtraction) shifts the data so its mean is 0. In StandardScaler: z = (x – mean)/std achieves both zero-centring and unit variance. In neural networks, zero-centred inputs speed up convergence because gradients flow more symmetrically. Tanh activation is preferred over sigmoid partly because its output is zero-centred (range –1 to +1), unlike sigmoid (range 0 to 1) which always produces positive gradients causing zig-zag updates.

What is causality vs. correlation? Why is this distinction important in ML?

Correlation: two variables move together statistically (e.g., ice cream sales and drowning rates both rise in summer). Causality: one variable directly causes another. Correlation does NOT imply causation. In ML, models capture correlations in training data — they predict patterns but do not establish cause-effect. Confounding variables (like 'summer') can create spurious correlations. For business decisions, understanding causality (via A/B testing, causal inference) is essential beyond predictive modelling.

Is PCA linear or non-linear? What about Kernel PCA?

Standard PCA is LINEAR — it finds linear combinations of features (principal components) that maximise variance. It cannot capture non-linear structure. Kernel PCA uses the kernel trick to implicitly map data into a high-dimensional feature space and applies PCA there, effectively performing non-linear dimensionality reduction. Common kernels: RBF, polynomial, sigmoid. Use Kernel PCA when data has non-linear structure that linear PCA cannot capture.

Does bagging give ALL features to each tree in Random Forest?

No — this is a key distinction. In a basic Bagging ensemble (BaggingClassifier), each learner may use all features. But Random Forest adds an extra layer of randomness: at each split in each tree, only a RANDOM SUBSET of features is considered (typically sqrt(n_features) for classification, n_features/3 for regression). This decorrelates the trees further, reducing variance more than plain bagging. The max_features parameter controls this.

Why is Random Forest better than a single Decision Tree?

A single Decision Tree tends to overfit (high variance) — small changes in data produce very different trees. Random Forest reduces variance by: 1) Bootstrap sampling (each tree sees different data). 2) Feature randomness at each split (trees are decorrelated). 3) Averaging/voting over many trees cancels out individual errors. Result: RF has similar bias to a single deep tree but much lower variance, giving better generalisation.

Why are Bagging/Boosting better than a single Decision Tree?

Single DT: high variance (overfits) if deep, high bias (underfits) if shallow. Bagging: trains multiple DTs on bootstrap samples and averages — reduces variance, keeps bias similar. Boosting: trains DTs sequentially on residuals — reduces bias, slightly increases variance but net effect is lower total error. Both benefit from the wisdom of crowds: combining many weak/medium learners outperforms a single

learner.

Why should learning rate be 0.01, 0.1, 0.2 and NOT 1, 2, 3?

Learning rate η controls the step size in gradient descent. If $\eta \geq 1$, the steps are too large — you overshoot the minimum repeatedly and the loss oscillates or diverges. In boosting (XGBoost/LightGBM), η=1 means each tree gets full weight — no shrinkage — so the model memorises training data (overfits). Small η (0.01–0.2) means each tree contributes a small amount, requiring more trees but leading to better generalisation. The convention is: lower LR + more estimators = better, more regularised model.

At what value of k in KNN is there overfitting vs. underfitting?

Small k (e.g., k=1): model memorises training data, very sensitive to noise → overfitting (high variance, low bias). As k increases: model becomes smoother and more generalised. Large k (e.g., k=n): model predicts the majority class for everything → underfitting (high bias, low variance). Optimal k is found via cross-validation, typically odd values to avoid ties. Rule of thumb: start with k=sqrt(n_samples).

If a dataset size is increased, which model would perform well?

Models that scale well with more data: Deep Learning (neural networks) — benefits most from large data. Gradient Boosting (XGBoost/LightGBM) — continues to improve. Logistic Regression — also scales well but hits a ceiling (linear model). Models that struggle: KNN — prediction time O(n), becomes very slow. SVM with RBF kernel — $O(n^2)$ to $O(n^3)$ training complexity. General rule: more data helps all models, but neural networks and boosting benefit most.

Can you solve a regression problem as a classification problem?

Yes, by binning the continuous target into discrete buckets (e.g., price ranges: low/medium/high). This trades some precision for interpretability. The reverse (classification as regression) is also possible — treat class labels as numbers and threshold the output, though log-odds modelling (logistic regression) is more principled. The examiner's specific question: 'can you solve this regression problem using classification?' — answer yes, by discretising the target variable into meaningful ranges.

Is data imbalanced or balanced — how do you know?

Check class distribution: df['target'].value_counts() or df['target'].value_counts(normalize=True). Rule of thumb: if the minority class is < 10–20% of the dataset, consider it imbalanced. Visual: bar plot of class counts. In the viva context, check your training data's class distribution before and after train_test_split. If imbalanced, use stratify=y in split, class_weight='balanced' in models, SMOTE, or appropriate metrics (F1, AUC) instead of accuracy.

What is the ROC-AUC curve and why plot it if F1-score is already plotted?

ROC curve and F1-score answer different questions. F1-score is computed at a FIXED threshold (usually 0.5) and summarises precision-recall balance at that point. ROC-AUC evaluates performance across ALL possible thresholds — it shows how well the model ranks positives above negatives regardless of threshold choice. Use both: F1 for operational threshold performance; AUC for overall discriminative ability. PR-AUC (Precision-Recall AUC) is better than ROC-AUC for severely imbalanced datasets.

What is autocorrelation in time series?

Autocorrelation measures the correlation of a time series with its own past values (lagged versions of itself). ACF (AutoCorrelation Function) plots correlation at each lag. PACF (Partial ACF) shows direct correlation with each lag after removing effects of intermediate lags. Used to: determine if a time series is stationary, choose ARIMA parameters (p from PACF, q from ACF), and detect seasonality patterns.

What is a rolling mean and why use it?

A rolling mean (moving average) computes the average of a fixed-size window sliding over time. E.g., 7-day rolling mean smooths daily noise to reveal weekly trends. Used as a feature in time series ML to capture recent trend without using future data (no leakage). Also used to detect non-stationarity and as a baseline for anomaly detection.

What are all types of feature reduction techniques?

Feature SELECTION (keeps original features): Filter methods (correlation, chi-square, mutual information, variance threshold). Wrapper methods (RFE, forward/backward selection). Embedded methods (L1/Lasso regularisation, tree feature importance). Feature EXTRACTION (creates new features): PCA, Kernel PCA, LDA, TruncatedSVD/LSA, t-SNE, UMAP (for visualisation), Autoencoders (deep learning). Dimensionality reduction is the umbrella term covering both.

What is encoding for datetime/temporal features in time series?

Datetime objects cannot be fed directly to ML models. Extract: year, month, day, hour, minute, day_of_week (0=Monday), day_of_year, week_of_year, is_weekend, is_holiday, quarter. For cyclical features (hour, month, day_of_week) use sin/cos encoding to preserve cyclical nature: sin(2π × hour/24) and cos(2π × hour/24) — so that hour 23 is close to hour 0. Also add lag features and rolling statistics.

How do you compare time taken by two models?

Use Python's time module or %%timeit magic in Jupyter: import time start = time.time() model.fit(X_train, y_train) end = time.time() print(f'Training time: {end-start:.2f}s') Or use timeit. For sklearn, you can also check fit_time_ from cross_validate(). Key insight: SVM O(n²–n³), KNN O(1) train but O(n) predict, tree models O(n log n), neural networks depend on architecture and epochs.

```
import time

# Model 1
start = time.time()
model1.fit(X_train, y_train)
time1 = time.time() - start

# Model 2
start = time.time()
model2.fit(X_train, y_train)
time2 = time.time() - start

print(f'Model1: {time1:.3f}s, Model2: {time2:.3f}s')
```

What is the difference between authentication and authorization?

Authentication: verifying WHO you are (identity verification — login with username/password, OTP, biometrics). Example: checking your student ID at the viva. Authorization: verifying WHAT you are allowed to do (permissions — after login, can you access admin panel?). Example: after ID verification, can you submit to this form? Auth flow: Authenticate first → then Authorize. JWT tokens handle both in modern APIs.

DECISION TREES — DEEP DIVE QUESTIONS

Explain the decision tree building process step by step.

1) Start with all training data at the root node. 2) For each feature, evaluate all possible split thresholds. 3) Compute impurity reduction (Information Gain = Parent Impurity – weighted child impurities) for each feature-threshold combination. 4) Choose the feature and threshold with the highest Information Gain as the split. 5) Recursively repeat for each child node. 6) Stop when: node is pure (all one class), max_depth is reached, node has fewer samples than min_samples_split, or Information Gain < threshold. 7) Remaining nodes become leaves; assign majority class (classification) or mean (regression).

What is the relationship between information gain and entropy?

Entropy H(S) = –Σ p■·log■(p■) measures impurity of set S. Information Gain IG(S, A) = H(S) – Σ (|S■|/|S|)·H(S■) where the sum is over all values v of attribute A, and S■ is the subset where A=v. IG measures how much entropy is REDUCED by splitting on attribute A. High IG = attribute A separates classes well = good split. Gini Index is an alternative impurity measure (doesn't use log, slightly faster).

What is the range of the Gini index and how is it calculated?

Gini = 1 – Σ p■² where p■ is the fraction of samples belonging to class i. Range: 0 (pure node, all one class) to 0.5 (binary, maximum impurity, 50/50 split). For multi-class with c classes: max Gini = 1 – 1/c (when classes are equally distributed). Lower Gini = purer node. Decision Tree minimises weighted Gini of child nodes. Gini vs Entropy: both work similarly; Gini is faster (no log), Entropy can be slightly more informative for multi-class.

What is the height vs. depth of a decision tree? Binary tree node count?

Depth of the tree = number of edges on the longest path from root to leaf = number of levels – 1. Height of a node = length of longest path from that node to a leaf. Root has the maximum height = tree depth. Leaves have height = 0. For a FULL binary tree of depth d: maximum nodes = 2^(d+1) – 1. E.g., depth=3: max nodes = 2■ – 1 = 15. max_depth in sklearn = maximum depth allowed (limits tree growth to prevent overfitting).

What is the difference between a Tree and a Graph?

A Graph is a general structure: a set of nodes (vertices) connected by edges. Edges can form cycles, nodes can have any number of connections, no hierarchy required. A Tree is a special CONNECTED, ACYCLIC graph with a hierarchical structure: one root node, each node (except root) has exactly one parent, no cycles. In ML: Decision Trees are trees (hierarchical, acyclic). Neural networks with skip connections (ResNets) form DAGs (directed acyclic graphs). Knowledge graphs are general graphs.

RANDOM FOREST & ENSEMBLE — SPECIFIC QUESTIONS

Is RandomForestClassifier a bagging or boosting algorithm? How?

Bagging (Bootstrap Aggregating). Random Forest trains N trees INDEPENDENTLY and IN PARALLEL on bootstrap samples (random samples with replacement). Predictions are combined by majority vote (classification) or mean (regression). It is NOT boosting — boosting trains sequentially and corrects errors of previous models. Random Forest = Bagging + extra feature randomness (only sqrt(features) considered at each split).

What is the loss function in Random Forest Classifier?

Random Forest itself does not minimise a single differentiable loss function like gradient-based models. Each tree is built using impurity measures: Gini index or Entropy (Information Gain). The forest makes predictions by aggregating tree votes. There is no explicit gradient-based optimisation — RF is a non-parametric method. This distinguishes it from gradient boosting which explicitly minimises a loss function via gradient descent in function space.

What are the parameters of RandomForestClassifier and what do they do?

n_estimators: number of trees (more = more stable, more compute). max_depth: maximum tree depth (None = grow until pure). min_samples_split: minimum samples to consider splitting a node. min_samples_leaf: minimum samples required in a leaf. max_features: features to consider per split ('sqrt', 'log2', int, float). bootstrap: use bootstrap sampling (True = RF; False = Pasting). class_weight: handle imbalance. oob_score: use out-of-bag samples for evaluation. random_state: for reproducibility.

What are the regularisation methods for Random Forest?

RF doesn't have explicit L1/L2 regularisation, but complexity is controlled via: max_depth (limit tree depth). min_samples_split & min_samples_leaf (require minimum samples to split or be a leaf — prevents over-splitting small node). max_features (feature randomness reduces overfitting). n_estimators (more trees = more stable but not more regularised per se). min_impurity_decrease (only split if impurity decreases by this amount).

How does gradient boosting build 10 weak learners sequentially? Explain the process.

Step 1: Initialise with a simple prediction (e.g., mean of target for regression). Step 2: Compute pseudo-residuals = negative gradient of loss w.r.t. current prediction. Step 3: Train a shallow decision tree (weak learner) to predict these residuals. Step 4: Update prediction: F_m(x) = F_{m-1}(x) + η × tree_m(x), where η = learning rate. Step 5: Repeat steps 2–4 for m = 1 to 10 (or n_estimators). Each tree corrects the previous model's mistakes. Shrinkage (η < 1) prevents overfitting by making small

incremental updates.

LOGISTIC REGRESSION — SPECIFIC QUESTIONS

What is the penalty parameter in Logistic Regression? What are the options?

penalty controls regularisation type: 'l2' (default): Ridge regularisation — shrinks all weights, rarely eliminates them. 'l1': Lasso regularisation — can zero out weights (feature selection). 'elasticnet': combination of L1 and L2 (requires solver='saga'). 'none': no regularisation (may overfit on small datasets). The strength of regularisation is controlled by C (inverse: smaller C = more regularisation).

What are the different solvers in Logistic Regression and when to use each?

'lbfgs' (default): L-BFGS quasi-Newton method. Supports L2 and none. Good for small-medium datasets, multiclass. 'liblinear': coordinate descent. Supports L1 and L2. Good for small datasets, only binary OvR. 'saga': Stochastic Average Gradient Amended. Supports all penalties including elasticnet. Best for large datasets. 'sag': Stochastic Average Gradient. Supports L2 and none. Faster on large datasets. 'newton-cg': Newton's method with conjugate gradient. L2 and none only. Rule: use 'lbfgs' by default; 'saga' for L1/elasticnet or large data; 'liblinear' for small binary tasks.

What is the SGDClassifier vs. Logistic Regression in sklearn?

SGDClassifier with loss='log_loss' is equivalent to Logistic Regression but trained with Stochastic Gradient Descent — better for very large datasets (online learning). SGDClassifier with loss='hinge' is equivalent to a linear SVM. SGDClassifier with loss='modified_huber' is a robust version of log loss. Key difference: SGDClassifier updates weights one sample (or mini-batch) at a time, making it memory-efficient. LogisticRegression uses more sophisticated optimisers (L-BFGS, SAG) that converge more reliably on smaller datasets.

SVM — SPECIFIC QUESTIONS

What is the 'degree' parameter in SVC (polynomial kernel)?

For kernel='poly', the polynomial kernel is K(x, y) = (γ·x■y + r)^d, where d = degree. degree=2: quadratic decision boundary (can model ellipses, parabolas). degree=3: cubic boundary. Higher degree = more complex, more risk of overfitting. Default is degree=3. For linearly separable data, use kernel='linear'. RBF kernel is generally preferred over polynomial as it has fewer hyperparameters.

What is the 'C' parameter in SVC and how does it affect the model?

C is the regularisation parameter (inverse of regularisation strength). Large C: hard margin — tries to classify ALL training points correctly, narrow margin, complex boundary → overfitting risk. Small C: soft margin — allows some misclassifications (slack variables), wider margin, simpler boundary → better generalisation. Tune C with cross-validation. Combine with kernel and gamma for full tuning.

What is the MNIST pipeline question commonly asked in Level 2 vivas?

A commonly asked coding question: load MNIST, split 50:50 with random_state=42, create a Pipeline with StandardScaler + SVM, fit and print classification report. Then redo with PCA added before SVM and compare scores.

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.metrics import classification_report

mnist = fetch_openml('mnist_784', version=1, as_frame=False)
X, y = mnist.data, mnist.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42)

# Without PCA
pipe1 = Pipeline([('scaler', StandardScaler()), ('svm', SVC())])
pipe1.fit(X_train, y_train)
print(classification_report(y_test, pipe1.predict(X_test)))

# With PCA
pipe2 = Pipeline([('scaler', StandardScaler()),
                  ('pca', PCA(n_components=100)),
                  ('svm', SVC())])
pipe2.fit(X_train, y_train)
print(classification_report(y_test, pipe2.predict(X_test)))
```

REINFORCEMENT LEARNING & ML TYPES

What is Reinforcement Learning and give examples?

Reinforcement Learning (RL) is a type of ML where an AGENT learns to make decisions by interacting with an ENVIRONMENT. The agent receives REWARDS for good actions and PENALTIES for bad ones, and learns a POLICY (strategy) to maximise cumulative reward. No labelled dataset needed — the agent explores and exploits. Examples: game-playing AI (AlphaGo, chess engines), robot navigation, recommendation systems, autonomous driving, trading algorithms. Key terms: state, action, reward, policy, value function, Q-learning, policy gradient.

Explain all types of Machine Learning with examples.

Supervised Learning: model learns from labelled (X, y) data. Classification: predict category (spam/not-spam, disease/no-disease). Regression: predict continuous value (house price, stock price). Unsupervised Learning: finds patterns in unlabelled data. Clustering: group similar points (KMeans, DBSCAN, Hierarchical). Dimensionality Reduction: PCA, t-SNE, Autoencoders. Association: find rules (Apriori — market basket analysis). Semi-supervised: small labelled + large unlabelled (e.g., self-training). Self-supervised: create labels from data itself (e.g., BERT predicts masked words). Reinforcement Learning: agent learns via reward/penalty feedback.

COLLINEARITY & MULTICOLLINEARITY

What is collinearity? If two columns are highly correlated, what should you do?

Collinearity: two features are highly linearly correlated (e.g., height in cm and height in inches). Multicollinearity: three or more features are linearly related. Problems: inflates variance of coefficient estimates in linear models, makes interpretation unreliable, can cause numerical instability. Detection: correlation matrix (|r| > 0.8–0.9), Variance Inflation Factor (VIF > 5–10). Solutions: 1) Drop one of the correlated features. 2) PCA (combines them). 3) Ridge Regression (L2 handles collinearity gracefully). 4) Create a combined feature (ratio or sum). Tree-based models are generally robust to collinearity.

SPECIFIC CODING QUESTIONS FROM VIVAS

Code: Load iris dataset and fit Logistic Regression on the ENTIRE dataset (no train-test split).

Sometimes asked to verify you know the sklearn API without necessarily needing a split.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
```

```
iris = load_iris()
X, y = iris.data, iris.target

lr = LogisticRegression(max_iter=200)
lr.fit(X, y)
print('Classes:', lr.classes_)
print('Coefficients shape:', lr.coef_.shape)
print('Training accuracy:', lr.score(X, y))
```

Code: Load iris dataset and fit SGDClassifier on the entire dataset.

Tests knowledge of SGDClassifier as an alternative to LogisticRegression.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X, y = iris.data, iris.target

# SGD needs scaling
X_scaled = StandardScaler().fit_transform(X)

sgd = SGDClassifier(loss='log_loss', random_state=42, max_iter=1000)
sgd.fit(X_scaled, y)
print('Accuracy:', sgd.score(X_scaled, y))
```

Code: Append the same dataset to itself (double the rows). Stack datasets vertically.

Tests numpy/pandas stacking operations. Expected size: (300, 4) for iris (150 rows → 300).

```
import numpy as np
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
X = iris.data  # shape (150, 4)

# Method 1: numpy vstack
X_doubled = np.vstack([X, X])
print('Shape after vstack:', X_doubled.shape)  # (300, 4)

# Method 2: pandas concat
df = pd.DataFrame(X)
df_doubled = pd.concat([df, df], ignore_index=True)
print('Shape after concat:', df_doubled.shape)  # (300, 4)
```

Code: Create a random matrix of shape (1000, 5) with values between 0 and 1.

Basic numpy random generation test.

```
import numpy as np

# Values uniformly distributed in [0, 1)
matrix = np.random.rand(1000, 5)
print('Shape:', matrix.shape)   # (1000, 5)
print('Min:', matrix.min())     # ~0
print('Max:', matrix.max())     # ~1
```

```
print('First row:', matrix[0])

# If you need exact range [0, 1]
# np.random.uniform(0, 1, size=(1000, 5))
```

Code: Create a cat_df with only object dtype columns, then apply OHE for ≤10 unique values and target encoding for >10.

Tests advanced pandas + sklearn encoding logic.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Step 1: Create cat_df with object columns only
cat_df = df.select_dtypes(include='object').copy()

# Step 2: Separate by unique value count
low_card = [c for c in cat_df.columns if cat_df[c].nunique() <= 10]
high_card = [c for c in cat_df.columns if cat_df[c].nunique() > 10]

print('OHE columns:', low_card)
print('Target encode columns:', high_card)

# OHE for low cardinality
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
ohe_result = pd.DataFrame(
    ohe.fit_transform(cat_df[low_card]),
    columns=ohe.get_feature_names_out(low_card)
)

# Target encoding for high cardinality (manual mean encoding)
for col in high_card:
    means = df.groupby(col)['target'].mean()
    cat_df[col + '_te'] = cat_df[col].map(means)
```

Code: Change TF-IDF Vectorizer to only produce trigrams (3-word sequences).

Change ngram_range parameter so only trigrams (n=3 to n=3) are produced.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Unigrams only: ngram_range=(1, 1)
# Unigrams + bigrams: ngram_range=(1, 2)
# Trigrams only: ngram_range=(3, 3)
# Range 6-10: ngram_range=(6, 10)  <- what it is called: 6-grams to 10-grams

vectorizer = TfidfVectorizer(
    ngram_range=(3, 3),   # ONLY trigrams
    max_features=50000
)
X_tfidf = vectorizer.fit_transform(corpus)
print('Vocabulary size:', len(vectorizer.vocabulary_))
print('Feature matrix shape:', X_tfidf.shape)
```

Code: Write IQR-based outlier detection and treatment manually.

Common coding question on outlier handling.

```
import pandas as pd
import numpy as np

def detect_outliers_iqr(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f'{col}: Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}')
    print(f'Lower bound: {lower:.2f}, Upper bound: {upper:.2f}')
    print(f'Outlier count: {len(outliers)}')
    return lower, upper

# Clip (Winsorise) outliers
def clip_outliers(df, col):
    lower, upper = detect_outliers_iqr(df, col)
    df[col] = df[col].clip(lower, upper)
    return df
```

Code: Print recall score for a model (instead of accuracy_score).

Tests knowledge of sklearn.metrics API.

```
from sklearn.metrics import recall_score, precision_score, f1_score

y_pred = model.predict(X_test)

# Binary
recall = recall_score(y_test, y_pred)
print('Recall:', recall)

# Multi-class
recall_macro = recall_score(y_test, y_pred, average='macro')
recall_weighted = recall_score(y_test, y_pred, average='weighted')
print('Recall (macro):', recall_macro)
print('Recall (weighted):', recall_weighted)
```

Code: Random Forest Classifier with GridSearchCV hyperparameter tuning.

Full implementation of RFC with GridSearchCV.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5, 10],
    'max_features': ['sqrt', 'log2']
}

rfc = RandomForestClassifier(random_state=42)
grid = GridSearchCV(
    rfc, param_grid,
    cv=5,
    scoring='f1_macro',
```

```
    n_jobs=-1,
    verbose=1
)
grid.fit(X_train, y_train)

print('Best params:', grid.best_params_)
print('Best CV score:', grid.best_score_)
print('Test score:', grid.score(X_test, y_test))
```

Code: Load Decision Tree and plot the tree.

Visualise a fitted decision tree.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)

dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
print('Test accuracy:', dt.score(X_test, y_test))

plt.figure(figsize=(15, 8))
plot_tree(dt, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.title('Decision Tree')
plt.show()
```

Code: StandardScaler on the target variable (y). When and why?

Scaling the target y is useful in regression when the target has a large range/skew. Never scale y for classification. After prediction, inverse-transform to get original scale.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Only for regression tasks
y_scaler = StandardScaler()
y_train_scaled = y_scaler.fit_transform(y_train.reshape(-1, 1)).ravel()
y_test_scaled = y_scaler.transform(y_test.reshape(-1, 1)).ravel()

# Train model on scaled y
model.fit(X_train, y_train_scaled)
y_pred_scaled = model.predict(X_test)

# Inverse transform to get original scale
y_pred_original = y_scaler.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()
print('Predictions (original scale):', y_pred_original[:5])
```

Code: Change HPT from RandomizedSearchCV to GridSearchCV in a notebook.

Know how to switch between the two — same param_grid, different class.

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import GradientBoostingClassifier
```

```
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.05, 0.1]
}

# RandomizedSearchCV (original)
# rs = RandomizedSearchCV(GBC(), param_grid, n_iter=10, cv=5)

# Switch to GridSearchCV (change: remove n_iter, use GridSearchCV)
grid = GridSearchCV(
    GradientBoostingClassifier(random_state=42),
    param_grid,   # same param_grid works for both
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)
grid.fit(X_train, y_train)
print('Best:', grid.best_params_)
```

TRICKY / EXAMINER-FAVOURITE QUESTIONS

Why random_state=42? What if the examiner says they are 'fed up' with the Hitchhiker's Guide answer?

The real answer: random_state is any integer seed for the random number generator — it ensures reproducibility (same results every run). 42 is a convention in the ML community borrowed from 'The Hitchhiker's Guide to the Galaxy' (42 = answer to everything). But if the examiner pushes: 'Any non-negative integer works. 42 is chosen by convention in the data science community for no mathematical reason — you could use 0, 1, 2023, or any number. The important property is that once set, results are reproducible for debugging, sharing, and paper replication.'

What is the XGBoost objective/metric parameter ('xgboost matrix')?

XGBoost's objective parameter specifies the learning task and loss function: 'binary:logistic': binary classification, outputs probability. 'multi:softmax': multi-class, outputs class index. 'multi:softprob': multi-class, outputs probabilities. 'reg:squarederror': regression with MSE. 'reg:logistic': regression with log loss. eval_metric specifies the evaluation metric used for validation: 'logloss', 'mlogloss', 'error', 'rmse', 'auc'. These are set in XGBClassifier(objective=..., eval_metric=...).

Why did you use only boosting/leaf-based models and not other models?

A justified answer: Boosting models (XGBoost, LightGBM) consistently outperform linear models on tabular data with complex non-linear relationships. They handle missing values internally, require minimal preprocessing (no scaling needed for trees), naturally capture feature interactions, and their leaf-wise growth (LightGBM) is computationally efficient. I also tried linear models (Logistic Regression) as a baseline but they performed significantly worse, confirming the non-linear nature of the data.

Is removing outliers always good practice? Can we do something else?

No — removing outliers is NOT always good practice. Outliers may be: 1) Valid extreme values (e.g., very high income earners — important for the model). 2) Informative patterns (fraud transactions are by definition 'outliers'). 3) Rare but real events in the tail distribution. Alternatives to removal: Capping/Winsorising (clip to 1st/99th percentile). Robust scaling (RobustScaler uses IQR). Log transformation (compresses extreme values). Robust models (tree-based models are naturally robust to outliers). Separate model for outlier segment. Only remove if the outlier is a data entry error (e.g., age = 999).

Why did you plot an ROC curve if you are already using F1-score? Are they redundant?

No, they are complementary. F1-score is computed at a SINGLE threshold (0.5 by default) and tells you performance at that operating point. ROC-AUC evaluates performance ACROSS ALL thresholds — useful when you haven't decided on the optimal threshold. In production, you may tune the threshold based on business requirements (e.g., lower threshold for high-recall fraud detection). ROC-AUC also gives a threshold-independent view of discriminative power. So: use F1 to evaluate at your chosen threshold, AUC to evaluate the model's overall ranking ability.

What is a multi-class confusion matrix — how do you identify TP, TN, FP, FN for each class?

For an N-class confusion matrix (rows=actual, cols=predicted), for each class k: TP_k = cm[k, k] (correctly predicted as class k). FP_k = sum(cm[:, k]) – cm[k, k] (predicted as k but actually other classes). FN_k = sum(cm[k, :]) – cm[k, k] (actually k but predicted as other classes). TN_k = total_samples – TP_k – FP_k – FN_k (everything else correctly not-k). Precision_k = TP_k / (TP_k + FP_k). Recall_k = TP_k / (TP_k + FN_k).

What is the 'cv' parameter in GridSearchCV/RandomizedSearchCV — exactly what does it do?

cv specifies the cross-validation strategy: cv=5 (integer): uses 5-fold stratified CV for classification, regular for regression. Each fold: 4 folds for training, 1 fold for validation, rotated 5 times. cv=KFold(n_splits=5, shuffle=True): explicit control. cv=StratifiedKFold(n_splits=5): preserves class distribution per fold. More cv = more reliable estimate but more compute (cv=5 → 5x fits per combination). Total fits for GridSearchCV = n_combinations × cv. For RandomizedSearchCV = n_iter × cv.

In diabetes dataset, which metric is the best choice and why?

For diabetes prediction (binary: diabetic/not-diabetic): Recall (Sensitivity) is most important — missing a diabetic patient (False Negative) is dangerous (they don't get treatment). High Recall minimises FN. But very low Precision means too many false alarms (unnecessary anxiety, costs). Best choice: F1-score or F2-score (weights Recall higher than Precision). Also report AUC-ROC for threshold-independent evaluation. Avoid accuracy if data is imbalanced (most people are non-diabetic).

What does '.head()' vs '.describe()' vs '.info()' show?

df.head(n): displays first n rows (default 5). Quick sanity check on data format. df.tail(n): last n rows. df.describe(): statistical summary of numeric columns: count, mean, std, min, Q1, median, Q3, max. df.info(): column names, non-null counts, dtypes, memory usage. Best for spotting missing values and wrong dtypes. df.shape: (rows, columns) tuple. df.dtypes: data type of each column.

TIME SERIES SPECIFIC QUESTIONS

What is ARIMA and how does it work? Explain its parameters p, d, q.

ARIMA (AutoRegressive Integrated Moving Average) is a time series forecasting model. p (AR order): number of lag observations included in the model (autoregressive part). Uses p past values to predict the next value. d (differencing order): number of times the series is differenced to make it stationary. If d=1: subtract consecutive values. q (MA order): number of lagged forecast errors in the prediction equation (moving average part). Uses q past errors to correct predictions. SARIMA adds seasonal terms (P, D, Q, s) for seasonal patterns. Determine p from PACF plot, q from ACF plot, d from ADF stationarity test.

What is SARIMA and when to use it over ARIMA?

SARIMA (Seasonal ARIMA) = ARIMA(p,d,q)(P,D,Q,s) where P,D,Q are seasonal counterparts of p,d,q and s is the seasonal period (e.g., s=12 for monthly data, s=7 for weekly). Use SARIMA when data shows clear seasonality (e.g., retail sales spike every December). ARIMA handles non-seasonal patterns; SARIMA handles both. Identify seasonality via: ACF/PACF plots (spikes at multiples of s), time series decomposition (seasonal_decompose), or visual inspection.

What is time series cross-validation (TSCV) and why can't you use regular k-fold?

Regular k-fold randomly shuffles data — this breaks temporal order, causing future data to leak into training (data leakage). TSCV (walk-forward validation): Fold 1: Train on [1..k], validate on [k+1..k+n].

Fold 2: Train on [1..2k], validate on [2k+1..2k+n]. Each fold only uses past to predict future. sklearn: TimeSeriesSplit(n_splits=5). This gives a realistic estimate of how the model will perform on genuinely unseen future data.

What is log transformation of the target variable and why use it?

If the target variable is right-skewed (many low values, few very high values), log transform (y_new = log(y+1)) compresses the range, making it more normally distributed. Benefits: reduces the effect of extreme outliers on MSE/RMSE loss, can improve linear model performance, predictions are more stable. After predicting log(y), apply inverse transform: y = exp(y_pred) – 1. Power transform (Box-Cox) is a generalisation: y^λ for optimal λ found by MLE. sklearn: PowerTransformer(method='box-cox') or FunctionTransformer(np.log1p).